



DEVELOPMENT DOCUMENT

Document 07 of 09 · B2B Data Engineering Platform



Technical design and implementation guide: environment, repository, configuration, component specifications, scripts, orchestration and runbook.

Document ID	B2B-DEP-07
Version	1.2
Status	Final
Date	05/10/2026
Prepared by	Syed Kazim Raza
Reviewed by	[Team Lead]

05/10/2026

DATAROPES.AI

Data Engineering Internship · B2B Data Engineering Platform

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	-	Syed Kazim Raza	Template created
1.0	28/09/2026	Syed Kazim Raza	Development documentation of the completed platform
1.1	30/09/2026	Syed Kazim Raza	New settings and functions (dirty data, clear_staging); load_to_staging; milestone 30/09/2026
1.2	05/10/2026	Syed Kazim Raza	Formatting to documentation standard: table column widths, numbered figure and table captions with lists, figure descriptions, complete acronym list

Approvals

Role	Name	Signature	Date
Prepared by	Syed Kazim Raza		
Reviewed by	[Team Lead]		
Approved by	[Manager]		

Related Documents

Ref	Document	Location
R1	Business Requirements Document (BRD)	Provided by DataRopes
R2	Project Document (technical reference)	docs/deliverables/project_document.md
R3	Project Plan	docs/deliverables/project_plan.md
R4	README / Deployment Guide	README.md

Contents

Document Control	2
Contents	3
List of Figures.....	4
List of Tables.....	5
1. Introduction.....	6
1.1 Purpose.....	6
1.2 Scope	6
1.3 Intended Audience	6
1.4 Definitions & Acronyms.....	6
2. Development Environment	7
3. Repository Structure.....	7
4. Configuration.....	9
5. Coding Standards & Naming Conventions.....	9
6. Component Specifications.....	10
6.1 Data Generators (master, transaction, CDC).....	10
6.2 SQL Server Pipeline.....	10
6.3 Marketing Leads Pipeline	10
6.4 Web Logs Pipeline	10
6.5 Orchestrator (run_pipeline.py)	11
6.6 Utilities (logging, notifier, pipeline utils)	11
7. SQL Transformation Scripts (Staging → Intermediate)	11
8. Warehouse Load Scripts (Intermediate → Warehouse)	12
9. Marts / KPI Views	13
10. Orchestration & Execution Flow.....	13
11. Logging, Monitoring & Notifications	14
12. Build & Deployment Runbook	16
13. Version Control & Milestones	16
14. Known Limitations & Future Work.....	17

List of Figures

Figure 1: Repository structure (top-level folders) 8

Figure 2: Terminal after an incremental run — staging checked and cleared, warehouse loaded 15

Figure 3: metadata.pipeline_run_log — latest runs of every stage..... 16

List of Tables

Table 1: Intended Audience.....	6
Table 2: Definitions & Acronyms	6
Table 3: Development Environment.....	7
Table 4: Repository Structure.....	7
Table 5: Configuration	9
Table 6: Coding Standards & Naming Conventions.....	9
Table 7: Component specification — Data Generators (master, transaction, CDC)	10
Table 8: Component specification — SQL Server Pipeline	10
Table 9: Component specification — Marketing Leads Pipeline.....	10
Table 10: Component specification — Web Logs Pipeline.....	10
Table 11: Component specification — Orchestrator (run_pipeline.py)	11
Table 12: Component specification — Utilities (logging, notifier, pipeline utils).....	11
Table 13: SQL Transformation Scripts (Staging → Intermediate).....	11
Table 14: Warehouse Load Scripts (Intermediate → Warehouse).....	12
Table 15: Marts / KPI Views.....	13
Table 16: Orchestration & Execution Flow	13
Table 17: Build & Deployment Runbook	16
Table 18: Version Control & Milestones.....	16
Table 19: Known Limitations & Future Work	17

1. Introduction

1.1 Purpose

This document explains how the platform is implemented so that another engineer can build, run and extend it.

1.2 Scope

In scope:

- Python generators and pipelines, SQL transformation and load scripts, orchestration, logging, deployment

Out of scope:

- Architecture rationale (Architecture Document)
- Test cases (Test Document)

1.3 Intended Audience

Table 1: Intended Audience

Audience	How they use this document
Data Engineers	Build, run and extend the code
Reviewers	Review implementation quality

1.4 Definitions & Acronyms

Table 2: Definitions & Acronyms

Term	Definition
B2B	Business-to-Business: companies selling to other companies.
BI	Business Intelligence: reporting and dashboards, here Power BI.
BRD	Business Requirements Document: here the Project Requirements Document provided by DataRopes.
CDC	Change Data Capture: extracting only the rows that changed since the last run (here: watermark on updated_at).
CSV	Comma-Separated Values: the text-file format of the marketing leads and web logs feeds.
CTE	Common Table Expression: a named sub-query (WITH ...) inside a SQL statement.
DDL	Data Definition Language: SQL that creates schemas, tables, indexes and views.
DEP	Data Engineering Platform: prefix of the document IDs (B2B-DEP-01 ...).
ELT	Extract, Load, Transform: raw data is loaded first and transformed inside the database with SQL.
Generator	Python module that creates synthetic source data (Faker + business rules).
GeoIP	IP geolocation: looking up the country and city of an IP address.
ID	Identifier.
KPI	Key Performance Indicator: a business measure shown on the dashboard (KPI-01 ... KPI-15).
ODBC	Open Database Connectivity: the driver standard Python uses to connect to SQL Server.
Orchestrator	run_pipeline.py: runs all generators, pipelines and SQL stages in order.
Pipeline	Python module that moves one source into the staging schema.
Run log	metadata.pipeline_run_log: one row per stage execution.
SCD	Slowly Changing Dimension: SCD1 overwrites changes, SCD2 keeps every version with validity dates.
SQL	Structured Query Language.
Watermark	Last loaded updated_at per SQL Server table (metadata.pipeline_watermarks).

2. Development Environment

Table 3: Development Environment

Component	Version	Purpose
Python	3.10.0	Generators, pipelines, orchestrator
SQL Server	2022 Express (16.0)	Source database b2b_source_db
PostgreSQL	18.4 (15+ required for MERGE)	Staging, intermediate, warehouse, marts, metadata
ODBC Driver	ODBC Driver 17 for SQL Server	pyodbc connection to SQL Server
Power BI Desktop	Current release	Dashboard on schema marts
Git	Current release	Version control
Python packages	pandas 2.2.3, SQLAlchemy 2.0.49, psycpg2-binary 2.9.12, pyodbc 5.3.0, Faker 40.36.0, python-dotenv 1.2.2, win10toast 0.9	Pinned in requirements.txt

3. Repository Structure

Table 4: Repository Structure

Folder	Contents
config/	config.py (volumes, dates, seed, paths, logging) and database.py (SQL Server and PostgreSQL engines from .env)
data/	Generated CSV files: marketing_leads/marketing_leads.csv, web_logs/web_logs.csv
python/generators/	master_generator.py, transaction_generator.py, cdc_generator.py, load_to_sqlserver.py
python/pipelines/	sql_server_pipeline.py, marketing_leads_pipeline.py, web_logs_pipeline.py, run_pipeline.py
python/utils/	constants.py, date_weighting.py, logger.py, notifier.py, pipeline_utils.py, seed_manager.py
sql_server/	source1/ddl-source1.sql (source schema), metadata/ddl-metadata.sql, exploration/dataset_validations.sql
postgresql/	metadata/, staging/, intermediate/ (DDL, transform, tests), warehouse/ (DDL, Load, PARTITIONS_SETUP, tests), marts/ (KPI1-15, marts_ddl.sql), tests/ (integration, KPI checks)
docs/, logs/	Documentation, images, dashboard; pipeline.log

Figure 1 shows the top-level folders of the repository.

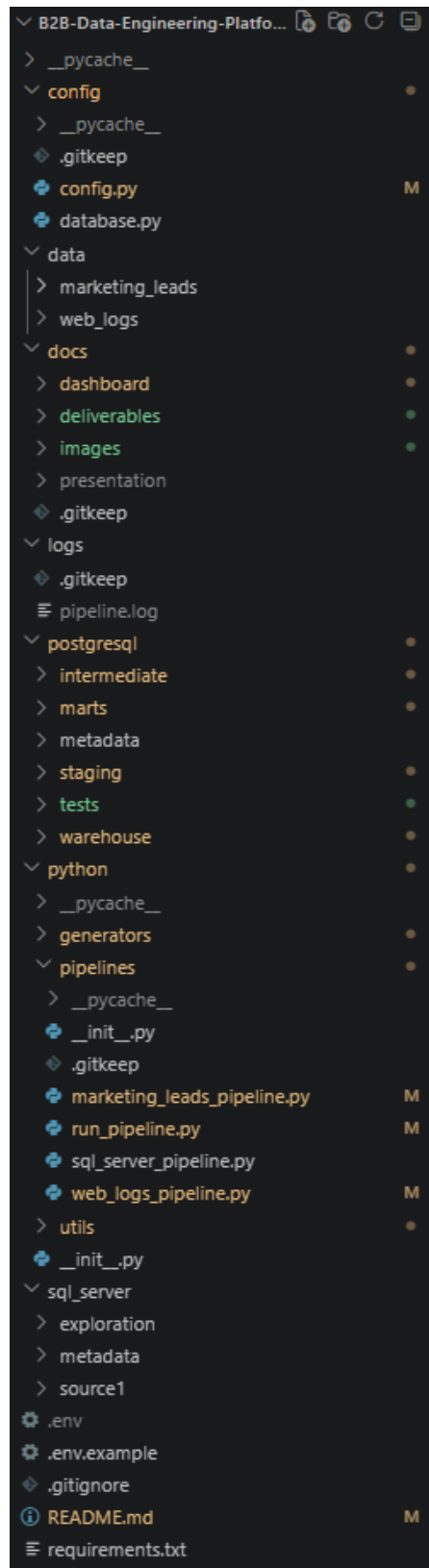


Figure 1: Repository structure (top-level folders)

4. Configuration

Table 5: Configuration

Parameter	File	Default	Description
NUM_COMPANIES / NUM_SUPPLIERS / NUM_CATEGORIES	config/config.py	2,500 / 400 / 10	Master data volumes
MIN/MAX_CUSTOMERS_PER_COMPANY	config/config.py	10 / 80	Customers per buyer company
MIN/MAX_SUPPLIERS_PER_PRODUCT	config/config.py	2 / 5	Suppliers offering each product
NUM_ORDERS / NUM_ORDER_ITEMS	config/config.py	150,000 / 450,000	Backfill transaction volumes
NUM_MARKETING_LEADS / NUM_WEB_LOGS	config/config.py	150,000 / 300,000	Backfill CSV volumes
START_DATE / END_DATE	config/config.py	today - 2 years / today	History window of the backfill
RANDOM_SEED	config/config.py	40	Deterministic seeding (with metadata.Generation_Seeds)
CHUNK_SIZE	config/config.py	2,500	Rows per batch in DataFrame.to_sql
DAILY_ORDERS_RANGE / DAILY_LEADS_RANGE / DAILY_WEB_LOGS_RANGE	config/config.py	(180, 230) / (180, 230) / (370, 450)	Volumes of one simulated day
CUSTOMER_ONBOARDING_MAX_DAYS / SALES_CYCLE_MAX_DAYS	config/config.py	90 / 90	Customers join within 90 days of their company; Won leads order within 90 days
DIRTY_DATA_PERCENT	config/config.py	1.5	% of CSV rows broken on purpose to test validation
SQL_SERVER_* / POSTGRES_* / POSTGRES_DATABASE	.env	-	Connection settings (not in Git)

5. Coding Standards & Naming Conventions

Table 6: Coding Standards & Naming Conventions

Area	Standard
Python	One module per responsibility; module docstring with execution flow; get_logger(__name__) for logging; try/except with logging and re-raise
SQL	One script per target table; CTE steps (latest -> cleaned -> validated) followed by MERGE; header comment explaining purpose and flow
Tables / columns	snake_case; dim_ / fact_ prefixes; <entity>_id business keys, <entity>_key surrogate keys; _loaded_at for technical columns
Scripts / files	Transform_<table>.sql, load_dim_<x>.sql / load_fact_<x>.sql, Test_<table>.sql / <table>_tests.sql, KPI<n>.sql
Commits	Short descriptive message per milestone

6. Component Specifications

6.1 Data Generators (master, transaction, CDC)

Table 7: Component specification — Data Generators (master, transaction, CDC)

Module / file	python/generators/master_generator.py, transaction_generator.py, cdc_generator.py, load_to_sqlserver.py
Purpose	Create realistic synthetic data: master data, 2 years of transactions, then one simulated day of new rows and updates per incremental run
Inputs	config.py volumes and dates, constants.py (countries, categories, bot user agents ...), existing SQL Server data for daily runs
Outputs	SQL Server source tables; marketing_leads.csv and web_logs.csv
Key functions	generate_master_datasets, generating_orders_and_orderitems_datasets, generate_web_logs, generate_marketing_leads, generate_daily_datasets, generate_daily_updates, add_dirty_leads, add_dirty_web_logs
Parameters	NUM_* volumes, START_DATE / END_DATE, RANDOM_SEED; weighted_datetime_between() adds seasonal and weekday patterns
Error handling	Exceptions are logged and re-raised so the orchestrator stops
Logging	pipeline.log (module logger)

6.2 SQL Server Pipeline

Table 8: Component specification — SQL Server Pipeline

Module / file	python/pipelines/sql_server_pipeline.py
Purpose	Incrementally copy the 8 SQL Server tables into staging
Inputs	source.* tables; watermarks from metadata.pipeline_watermarks
Outputs	staging.<table> (append); updated watermarks; run-log row
Key functions	extracting_incremental_data, profile_dataframe, load_to_staging, run
Parameters	TABLE_MAPPING, SOURCE1_QUERIES, CHUNK_SIZE
Error handling	Run logged as 'failed' with the error; watermarks not moved; exception re-raised
Logging	pipeline.log + metadata.pipeline_run_log (rows extracted/loaded, watermark)

6.3 Marketing Leads Pipeline

Table 9: Component specification — Marketing Leads Pipeline

Module / file	python/pipelines/marketing_leads_pipeline.py
Purpose	Reload data/marketing_leads/marketing_leads.csv into staging.marketing_leads (full snapshot)
Inputs	data/marketing_leads/marketing_leads.csv
Outputs	staging.marketing_leads; run-log row
Key functions	extract (pd.read_csv), profile, load_to_staging (TRUNCATE + to_sql append, one transaction), run
Parameters	Output path from config.py, CHUNK_SIZE
Error handling	Run logged as 'failed'; exception re-raised
Logging	pipeline.log + metadata.pipeline_run_log

6.4 Web Logs Pipeline

Table 10: Component specification — Web Logs Pipeline

Module / file	python/pipelines/web_logs_pipeline.py
Purpose	Reload data/web_logs/web_logs.csv into staging.web_logs (full snapshot)
Inputs	data/web_logs/web_logs.csv
Outputs	staging.web_logs; run-log row
Key functions	extract (pd.read_csv), profile, load_to_staging (TRUNCATE + to_sql append, one transaction), run
Parameters	Output path from config.py, CHUNK_SIZE
Error handling	Run logged as 'failed'; exception re-raised
Logging	pipeline.log + metadata.pipeline_run_log

6.5 Orchestrator (run_pipeline.py)

Table 11: Component specification — Orchestrator (run_pipeline.py)

Module / file	python/pipelines/run_pipeline.py
Purpose	Single entry point that runs the complete platform in the right order
Inputs	Command-line argument: initial or incremental
Outputs	Loaded intermediate and warehouse (staging emptied after use); run log; notification
Key functions	run_initial, run_incremental, run_module, run_transform, clear_staging, run_warehouse_load, main
Parameters	Ordered lists of transform and load scripts
Error handling	Any failure stops the run, logs it and shows a failure notification (exit code 1); intermediate stage in one transaction
Logging	pipeline.log, pipeline_run_log (intermediate_transform, warehouse_load), Windows toast

6.6 Utilities (logging, notifier, pipeline utils)

Table 12: Component specification — Utilities (logging, notifier, pipeline utils)

Module / file	python/utils/logger.py, notifier.py, pipeline_utils.py, seed_manager.py, date_weighting.py, constants.py
Purpose	Shared helpers for logging, notifications, watermarks and run log, seeding and realistic dates
Inputs	config.py settings
Outputs	Log lines, toast notifications, metadata rows
Key functions	get_logger, notify, get_last_watermark, update_watermark, log_run_start, log_run_end, get_live_seed, weighted_datetime_between
Parameters	LOG_LEVEL, LOG_FORMAT, PIPELINE_LOGS_PATH
Error handling	notify() never breaks a run (errors are caught and logged)
Logging	File + console handlers on logs/pipeline.log

7. SQL Transformation Scripts (Staging → Intermediate)

Table 13: SQL Transformation Scripts (Staging → Intermediate)

Order	Script	Source	Target	Logic Summary	Depends On
1	Transform_companies.sql	staging.companies	intermediate.companies	Latest version per key, trim/clean, validate, MERGE	-
2	Transform_categories.sql	staging.categories	intermediate.categories	Latest version per key, trim/clean, validate, MERGE	-
3	Transform_customers.sql	staging.customers	intermediate.customers	Latest version per key, trim/clean, validate, MERGE	companies
4	Transform_suppliers.sql	staging.suppliers	intermediate.suppliers	Latest version per key, trim/clean, validate, MERGE	companies
5	Transform_products.sql	staging.products	intermediate.products	Latest version per key, trim/clean, validate, MERGE	categories
6	Transform_supplier_product_mapping.sql	staging.supplier_product_mapping	intermediate.supplier_product_mapping	Latest version per key, trim/clean, validate, MERGE	suppliers, products

Order	Script	Source	Target	Logic Summary	Depends On
7	Transform_marketing_leads.sql	staging.marketing_leads	intermediate.marketing_leads	Save invalid rows in rejected_records, then latest version per key, trim/clean, validate, MERGE	-
8	Transform_orders.sql	staging.orders	intermediate.orders	Latest version per key, trim/clean, validate, MERGE; sets lead conversion_status	customers, companies, marketing_leads
9	Transform_order_items.sql	staging.order_items	intermediate.order_items	Latest version per key, trim/clean, validate, MERGE	orders, supplier_product_mapping
10	Transform_web_logs.sql	staging.web_logs	intermediate.web_logs	Save invalid rows in rejected_records, then latest version per key, trim/clean, validate, MERGE	-

8. Warehouse Load Scripts (Intermediate → Warehouse)

Table 14: Warehouse Load Scripts (Intermediate → Warehouse)

Order	Script	Target	Load Type	Logic Summary	Depends On
1	load_dim_date.sql	dim_date	Insert new dates	Calendar 2 years back to 6 months ahead	-
2	load_dim_companies.sql	dim_companies	SCD Type 2	Expire changed version, insert new current version	intermediate.companies
3	load_dim_customers.sql	dim_customers	SCD Type 1 MERGE	Overwrite changed attributes	intermediate.customers
4	load_dim_suppliers.sql	dim_suppliers	SCD Type 1 MERGE	Overwrite changed attributes	intermediate.suppliers
5	load_dim_products.sql	dim_products	SCD Type 1 MERGE	Adds category_name from categories	intermediate.products, categories
6	load_dim_supplier_product.sql	dim_supplier_product	SCD Type 1 MERGE	Adds supplier_key and product_key	dim_suppliers, dim_products
7	load_fact_orders.sql	fact_orders	MERGE	Keys looked up; company version valid at order date; item_count	dimensions, partitioning.sql

Order	Script	Target	Load Type	Logic Summary	Depends On
8	load_fact_order_items.sql	fact_order_items	MERGE	Keys looked up; order status copied from order	dimensions, partitioning.sql
9	load_fact_web_logs.sql	fact_web_logs	Insert new only	date_key from log timestamp	dim_date
10	load_fact_leads.sql	fact_leads	MERGE	date_key from lead date; order_id of converted leads	dim_date, intermediate.orders

9. Marts / KPI Views

Table 15: Marts / KPI Views

View	Source File	Reads From
marts.vw_monthly_revenue_trend	marts/KPI1.sql	fact_order_items, dim_date
marts.vw_revenue_by_company	marts/KPI2.sql	fact_order_items, dim_companies
marts.vw_revenue_by_product	marts/KPI3.sql	fact_order_items, dim_products
marts.vw_gross_margin_analysis	marts/KPI4.sql	fact_order_items, dim_products, dim_suppliers, dim_supplier_product
marts.vw_customer_lifetime_value	marts/KPI5.sql	fact_orders, fact_order_items, dim_companies
marts.vw_customer_activity	marts/KPI6.sql	fact_orders
marts.vw_geographic_sales	marts/KPI7.sql	fact_order_items, dim_companies
marts.vw_traffic_by_device	marts/KPI8.sql	fact_web_logs
marts.vw_geographic_web_traffic	marts/KPI9.sql	fact_web_logs
marts.vw_web_activity_quality	marts/KPI10.sql	fact_web_logs
marts.vw_lead_conversion_funnel	marts/KPI11.sql	fact_leads
marts.vw_lead_quality_vs_order_value	marts/KPI12.sql	fact_leads, fact_orders
marts.vw_supplier_revenue	marts/KPI13.sql	fact_order_items, dim_suppliers
marts.vw_supplier_product_performance	marts/KPI14.sql	fact_order_items, dim_products, dim_suppliers
vw_pipeline_execution_metrics, vw_pipeline_run_history, vw_pipeline_latest_run	marts/KPI15.sql	metadata.pipeline_run_log, pipeline_watermarks

10. Orchestration & Execution Flow

Table 16: Orchestration & Execution Flow

Step	Stage	Function / Command	Depends On	Logged As
1	Generate data	python.generators.* (initial: master + transactions; incremental: cdc_generator)	-	pipeline.log
2	SQL Server -> staging	python.pipelines.sql_server_pipeline	Step 1	sql_server_pipeline
3	Marketing leads -> staging	python.pipelines.marketing_leads_pipeline	Step 1	marketing_leads_pipeline

Step	Stage	Function / Command	Depends On	Logged As
4	Web logs -> staging	python.pipelines.web_logs_pipeline	Step 1	web_logs_pipeline
5	Staging -> intermediate	run_transform() - 10 scripts, one transaction	Steps 2-4	intermediate_transform
6	Clear staging	clear_staging() - check every row is in intermediate or rejected_records, then TRUNCATE	Step 5	pipeline.log
7	Intermediate -> warehouse	run_warehouse_load() - dimensions, partitioning.sql, facts	Step 6	warehouse_load
8	Notify	notify() - Windows toast success / failure	Step 7	pipeline.log

11. Logging, Monitoring & Notifications

- **logs/pipeline.log:** every module logs through get_logger() to the file and the console, format `time | level | module | line | message`.
- **metadata.pipeline_run_log:** each stage writes a row when it starts ('running') and updates it at the end ('completed' or 'failed') with rows extracted/loaded, watermark and error message.
- **metadata.pipeline_watermarks:** last loaded updated_at for each of the 8 SQL Server tables.
- **Pipeline health in Power BI:** KPI 15 views show load times, error rates, data freshness and watermark lag.
- **Notifications:** a Windows toast (win10toast) reports success or failure at the end of every run.

Figure 2 shows the terminal output of an incremental run: every stage completes, staging is checked and cleared and the warehouse is loaded.

```

D:\Courses\Data Engineer\Internship_DataRopes\DE-Internship-Project\B2B-Data-Engineering-Platform> python -m python.pipelines.run_pipeline incremental
026-09-30 21:52:07,039 | INFO | __main__ | 144 | Executing Transform_web_logs.sql
026-09-30 21:52:33,198 | INFO | __main__ | 158 | Transform_web_logs.sql completed successfully
026-09-30 21:52:33,198 | INFO | __main__ | 160 | All Intermediate transformations committed successfully.
026-09-30 21:52:33,210 | INFO | python.utils.pipeline_utils | 116 | [intermediate_transform] run c1c4dcdf30204026a2ed59e8f409a1ee -> completed (extracted rows=None, loaded rows=297930)
026-09-30 21:52:33,213 | INFO | __main__ | 191 | .....CHECKING AND CLEARING STAGING.....
026-09-30 21:52:34,366 | INFO | __main__ | 233 | Every staging row is in intermediate or rejected_records.
026-09-30 21:52:34,567 | INFO | __main__ | 241 | Staging tables cleared.
026-09-30 21:52:34,567 | INFO | __main__ | 407 | .....RUNNING WAREHOUSE LOAD.....
026-09-30 21:52:34,567 | INFO | __main__ | 247 | .....LOADING WAREHOUSE LAYER.....
026-09-30 21:52:34,569 | INFO | python.utils.pipeline_utils | 82 | [warehouse_load] run started, run_id = 28988806e6d7422db733c58bfcd2de5a
026-09-30 21:52:34,569 | INFO | __main__ | 278 | Executing partitioning.sql
026-09-30 21:52:34,630 | INFO | __main__ | 287 | partitioning.sql completed successfully.
026-09-30 21:52:34,630 | INFO | __main__ | 312 | Executing load_dim_date.sql
026-09-30 21:52:34,657 | INFO | __main__ | 328 | load_dim_date.sql completed successfully.
026-09-30 21:52:34,657 | INFO | __main__ | 312 | Executing load_dim_companies.sql
026-09-30 21:52:34,842 | INFO | __main__ | 328 | load_dim_companies.sql completed successfully.
026-09-30 21:52:34,842 | INFO | __main__ | 312 | Executing load_dim_customers.sql
026-09-30 21:52:36,414 | INFO | __main__ | 328 | load_dim_customers.sql completed successfully.
026-09-30 21:52:36,414 | INFO | __main__ | 312 | Executing load_dim_suppliers.sql
026-09-30 21:52:36,449 | INFO | __main__ | 328 | load_dim_suppliers.sql completed successfully.
026-09-30 21:52:36,449 | INFO | __main__ | 312 | Executing load_dim_products.sql
026-09-30 21:52:36,474 | INFO | __main__ | 328 | load_dim_products.sql completed successfully.
026-09-30 21:52:36,476 | INFO | __main__ | 312 | Executing load_dim_supplier_product.sql
026-09-30 21:52:36,508 | INFO | __main__ | 328 | load_dim_supplier_product.sql completed successfully.
026-09-30 21:52:36,508 | INFO | __main__ | 312 | Executing load_fact_orders.sql
026-09-30 21:52:44,853 | INFO | __main__ | 328 | load_fact_orders.sql completed successfully.
026-09-30 21:52:44,855 | INFO | __main__ | 312 | Executing load_fact_order_items.sql
026-09-30 21:53:06,089 | INFO | __main__ | 328 | load_fact_order_items.sql completed successfully.
026-09-30 21:53:06,089 | INFO | __main__ | 312 | Executing load_fact_web_logs.sql
026-09-30 21:53:30,609 | INFO | __main__ | 328 | load_fact_web_logs.sql completed successfully.
026-09-30 21:53:30,609 | INFO | __main__ | 312 | Executing load_fact_leads.sql
026-09-30 21:53:39,569 | INFO | __main__ | 328 | load_fact_leads.sql completed successfully.
026-09-30 21:53:41,896 | INFO | __main__ | 342 | Dimension tables clustered on their primary keys.
026-09-30 21:53:41,896 | INFO | __main__ | 344 | All Warehouse loads completed successfully.
026-09-30 21:53:41,900 | INFO | python.utils.pipeline_utils | 116 | [warehouse_load] run 28988806e6d7422db733c58bfcd2de5a -> completed (extracted rows=None, load
NDPROC return value cannot be converted to LRESULT
ypeError: WPARAM is simple, so must be an int object (got NoneType)
D:\Courses\Data Engineer\Internship_DataRopes\DE-Internship-Project\B2B-Data-Engineering-Platform>

```

Python

Pipeline Completed
Incremental pipeline run finished successfully.

Figure 2: Terminal after an incremental run — staging checked and cleared, warehouse loaded

Figure 3 shows the latest run of every stage in metadata.pipeline_run_log.

Query

Query History

1

2

3

4

5

6

SELECT

pipeline_name, status, rows_extracted, rows_loaded, run_started_at, run_ended_at

FROM metadata.pipeline_run_log

ORDER BY run_started_at DESC

LIMIT 10;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

Page No: 1

	pipeline_name character varying (100)	status character varying (20)	rows_extracted integer	rows_loaded integer	run_started_at timestamp without time zone	run_ended_at timestamp without time zone
1	warehouse_load	completed	[null]	1501	2026-09-30 17:32:26.594969	2026-09-30 17:33:26.293246
2	intermediate_transform	completed	[null]	297567	2026-09-30 17:31:42.741172	2026-09-30 17:32:25.471447
3	web_logs_pipeline	completed	301220	301220	2026-09-30 17:31:10.245685	2026-09-30 17:31:39.454006
4	marketing_leads_pipeli...	completed	150616	150616	2026-09-30 17:30:49.968991	2026-09-30 17:31:05.837898
5	sql_server_pipeline	completed	881	881	2026-09-30 17:30:43.129156	2026-09-30 17:30:44.729435
6	warehouse_load	completed	[null]	1498	2026-09-30 17:29:30.066425	2026-09-30 17:30:25.780681
7	intermediate_transform	completed	[null]	297165	2026-09-30 17:28:43.258518	2026-09-30 17:29:28.322206
8	web_logs_pipeline	completed	300806	300806	2026-09-30 17:28:16.144543	2026-09-30 17:28:39.986211
9	marketing_leads_pipeli...	completed	150413	150413	2026-09-30 17:27:57.621214	2026-09-30 17:28:11.602063
10	sql_server_pipeline	completed	851	851	2026-09-30 17:27:50.377578	2026-09-30 17:27:52.503623

Figure 3: metadata.pipeline_run_log — latest runs of every stage

12. Build & Deployment Runbook

Table 17: Build & Deployment Runbook

Step	Action	Command / Script
1	Install prerequisites	Python 3.10+, SQL Server Express + ODBC Driver 17, PostgreSQL 15+, Power BI Desktop
2	Install Python packages	pip install -r requirements.txt
3	Configure connections	Copy .env.example to .env and fill in SQL Server / PostgreSQL settings
4	Create PostgreSQL database	Empty database named as POSTGRESQL_DATABASE (b2b_warehouse_db)
5	Run SQL Server DDL	sql_server/source1/ddl-source1.sql, then sql_server/metadata/ddl-metadata.sql
6	Run PostgreSQL DDL	metadata, staging, intermediate, warehouse, partitioning.sql, marts_ddl.sql (in this order)
7	Initial load (2-year backfill)	python -m python.pipelines.run_pipeline initial
8	Daily incremental run	python -m python.pipelines.run_pipeline incremental
9	Validate and report	Run the SQL tests (0 rows expected); open docs/dashboard/dashboard.pbix or connect Power BI to schema marts

13. Version Control & Milestones

Table 18: Version Control & Milestones

Date	Commit / Tag	Milestone
2026-07-30	Initialize project structure	Project set-up, architecture and configuration

Date	Commit / Tag	Milestone
2026-08-05	transaction generator + SQL Server master data load	Source data generation
2026-08-06	CDC generator + 2 years backfill + continuous ingestion	Source layer complete
2026-08-07	ELT pipelines, staging schema populated	Staging layer complete
2026-08-11	Intermediate schema validated & populated	Intermediate layer complete
2026-08-15	Warehouse schema validated & populated	Warehouse layer complete, pipeline end-to-end
2026-08-17	Production data volumes, Power BI dashboard	Marts and dashboard complete
2026-09-30	Review fixes	Realistic data, quarantine table, temporary staging, KPI fixes

14. Known Limitations & Future Work

Table 19: Known Limitations & Future Work

Item	Description	Planned Action
Scheduling	Runs are started manually, as specified in the requirements.	Windows Task Scheduler or Apache Airflow
IP geolocation	Locations are generated with the synthetic data.	GeoIP lookup (e.g. MaxMind GeoLite2) for real client IPs
Raw data archive	Staging is emptied after each run; raw files are not archived.	Keep raw files in a data lake / cheap storage
Cloud deployment	Platform runs on one local machine.	Containerise and move to a managed PostgreSQL / cloud warehouse